SOFTWARE ENGINEERING · SEMESTER 5 · B.TECH COMPUTER ENGINEERING

UNIT 5

Software Coding & Testing

Coding Standards & Review · STLC · Testing Strategies · Black-Box & White-Box · Test Case Design

Silver Oak University | School of Technology, Design & Computer Application

WHERE WE'RE HEADED

Unit 5 Roadmap

Five connected stops — from writing clean code to proving it actually works.



1. Coding Phase

Standards, Guidelines & Code Review



2. STLC

Software Testing Life Cycle — 6 phases



3. Testing Strategies

Unit, Integration, System, Acceptance, Regression



4. Testing Techniques

Black-Box vs White-Box



5. Test Case Design

Building real, traceable test cases



WHY THIS UNIT MATTERS

$440 Million. Lost in 45 Minutes.

In 2012, trading firm Knight Capital deployed new code to production — but an old, dead test-flag (a leftover unused/untested code path) got accidentally triggered on live servers. Nobody caught it because the code was never properly reviewed or regression-tested before deployment.

The lesson: coding standards, code review, and disciplined testing aren't paperwork — they're what keeps software (and companies) alive.



45 minutes

for the bug to bankrupt the firm



$440M

lost in erroneous trades



1 dead code path

left in from a decade earlier



PART 1 OF 5

Coding Standards, Guidelines & Code Review

Turning design into disciplined, reviewable code

CODING PHASE

From Design to Code — Where We Are in the SDLC



Once a design (architecture + module design) is ready, developers translate it into source code using a chosen programming language. This unit focuses on two things that decide whether that code is trustworthy: how it's written (coding standards & guidelines) and how it's tested (STLC, strategies, techniques).

Key idea: Good coding practices reduce the number of bugs BEFORE testing even begins — testing then catches whatever slips through.

What Are Coding Standards?

"A set of rules and guidelines that developers agree to follow when writing source code — covering naming, formatting, structure, and documentation."



Purpose

- Make code readable by every team member — not just the author
- Keep style consistent across thousands of files
- Reduce bugs caused by ambiguous or careless code
- Cut the cost of future maintenance and onboarding



Typical Areas Covered by Coding Standards

Naming Conventions
camelCase, PascalCase, meaningful names

Indentation & Layout
consistent spacing, braces, line length

Comments & Docs
why, not just what — header comments

Error Handling
consistent try/catch, no silent failures

Why Coding Standards Matter

Readability

Any developer can pick up the code and understand it quickly.

Team Collaboration

Multiple developers can work on the same codebase without style clashes.

Easier Maintenance

Bugs are fixed faster when code follows a predictable structure.

Fewer Bugs

Consistent patterns (e.g. always closing resources) prevent whole classes of defects.

Better Code Reviews

Reviewers focus on logic, not arguing about formatting.

Faster Onboarding

New hires ramp up quickly on a consistent codebase.

Example: Bad Code vs Good Code

Same logic. One follows coding standards, one doesn't — notice the difference in naming, spacing, and clarity.



Without Standards

```
int a;
float b,c;
void f(int x,int y)
{
if(x>y){
a=x;
}else{
a=y;}
b=a*1.5;c=b-2;
return;
}
```



With Coding Standards

```
int maxValue;
float discountedPrice, finalPrice;

// Returns the larger of two order quantities
void calculateDiscount(int qtyA, int qtyB) {
    if (qtyA > qtyB) {
        maxValue = qtyA;
    } else {
        maxValue = qtyB;
    }
    discountedPrice = maxValue * 1.5;
    finalPrice = discountedPrice - 2;
}
```

Coding Guidelines — Best Practices

Guidelines are broader recommendations (not rigid rules) that lead to higher-quality, more maintainable code.

Keep Functions Small

One function should do one thing — easier to test and reuse.

Avoid Deep Nesting

Too many nested if/for blocks hurt readability; refactor early.

Comment the 'Why'

Explain intent and edge cases, not just restate the code.

Handle Errors Explicitly

Never swallow exceptions silently; log or propagate them.

Avoid Magic Numbers

Use named constants instead of unexplained literals like 86400.

DRY — Don't Repeat Yourself

Extract repeated logic into reusable functions/modules.

CLEARING THE CONFUSION

Coding Standards vs Coding Guidelines

Aspect	Coding Standards	Coding Guidelines
Nature	Mandatory rules	Recommended best practices
Flexibility	Little to no flexibility	Can be adapted per situation
Focus	Syntax, formatting, naming	Design quality, maintainability
Enforcement	Often enforced by linters/CI	Enforced through review & culture
Example	"Use camelCase for variables"	"Prefer small, single-purpose functions"

What Is Code Review?

"A systematic examination of source code by people other than the author, done to find bugs and improve quality before the code is merged."



Also called:

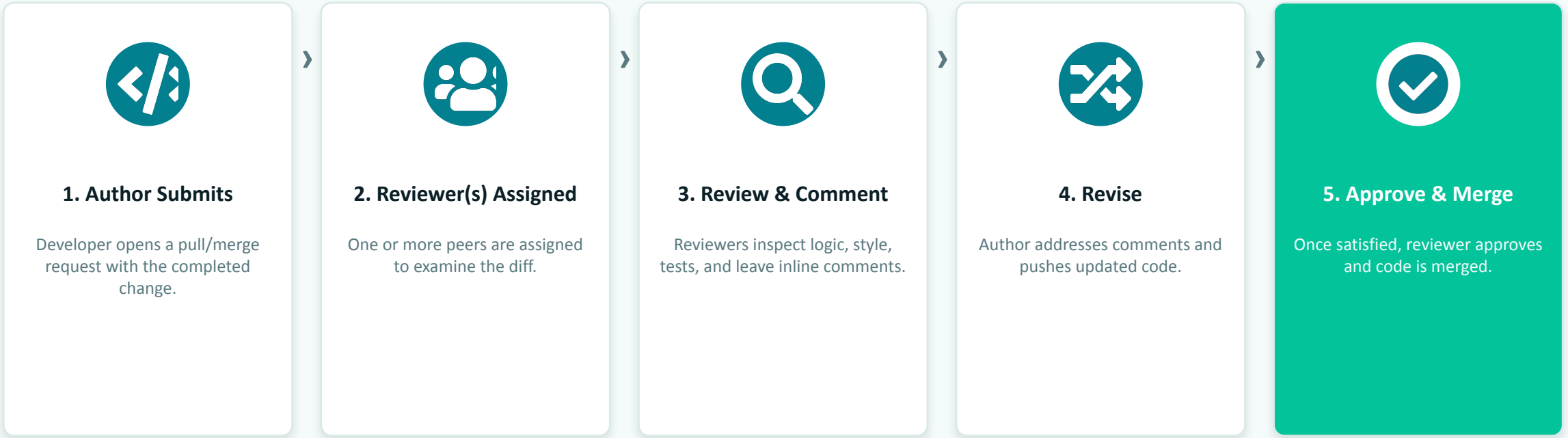
Peer Review

performed BEFORE code is compiled/executed — a form of static, human-driven testing.

What Reviewers Look For

- Logical correctness — does the code do what it's meant to do?
- Adherence to coding standards & guidelines
- Security issues (e.g. unvalidated input, hardcoded passwords)
- Performance concerns and unnecessary complexity
- Missing test coverage or edge cases

The Code Review Process



Types of Code Review

Formal Inspection

A structured, planned meeting (Fagan Inspection) with defined roles — moderator, author, reviewers — and a documented defect log.

Walkthrough

Author leads teammates through the code, explaining logic; informal, good for knowledge sharing.

Peer / Pair Review

Two developers write & review code together in real time, or one reviews a colleague's finished commit.

Tool-Assisted Review

Automated static analysis tools (SonarQube, ESLint) flag issues before/alongside human review.

A Real Code Review Checklist

Excerpt of what a reviewer actually checks before approving a pull request:



Does the code compile and pass all existing unit tests?



Are variable/function names meaningful and standard-compliant?



Is there dead code, commented-out blocks, or console.log() left in?



Are edge cases (null, empty input, zero) handled?



Is sensitive data (passwords, API keys) hardcoded anywhere?



Are new functions covered by new test cases?

Benefits & Challenges of Code Review



Benefits

- Catches bugs earlier and cheaper than testing/production
- Spreads code knowledge across the team
- Enforces consistent coding standards
- Mentors junior developers through feedback
- Improves overall design decisions



Challenges

- Time-consuming — slows down delivery if overdone
- Can create friction/ego conflicts between developers
- Reviewer may lack context and miss real issues
- Risk of becoming a rubber-stamp if rushed
- Needs tooling + culture to work well long-term



PART 2 OF 5

Software Testing Life Cycle (STLC)

The disciplined process behind every good test effort

What Is Software Testing?

> *"The process of executing a program with the intent of finding errors — and building confidence that it does what it's supposed to do."*

Goals of Testing



Find Defects

Uncover errors before the user does.



Build Confidence

Prove the software meets requirements.



Assess Quality

Measure reliability, performance & usability.



Prevent Regressions

Ensure new changes don't break old features.

Verification vs Validation (V&V)



Verification

"Are we building the product right?"

- Checks the process — reviews, walkthroughs, inspections
- Done without executing the code (static)
- Confirms design matches requirements
- Example: reviewing an SRS document for consistency



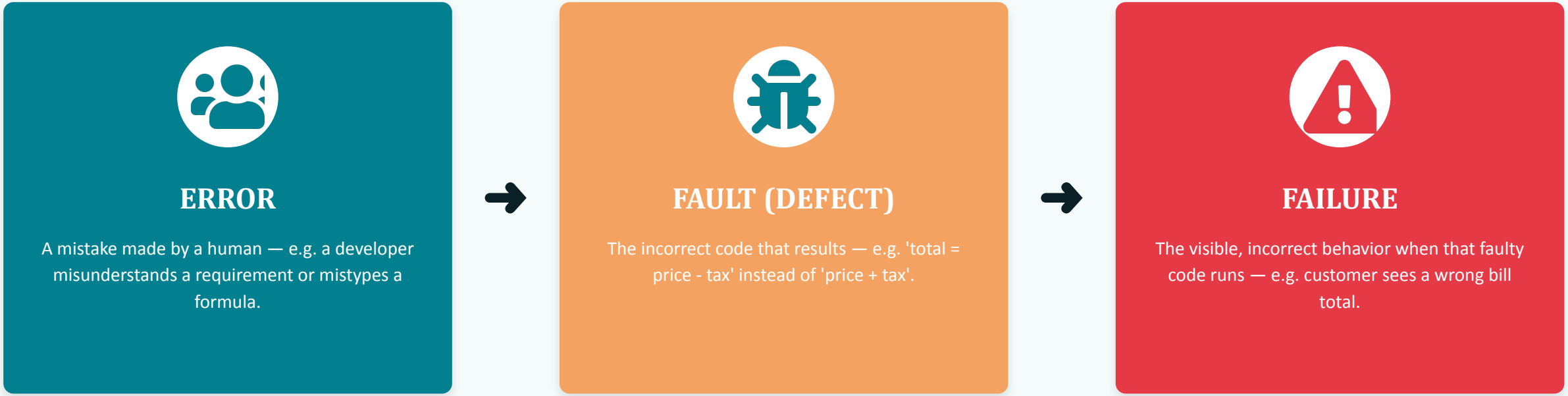
Validation

"Are we building the right product?"

- Checks the actual product — by executing it
- Done by running test cases (dynamic)
- Confirms the software meets user needs
- Example: running test cases on the login module

KEY TERMINOLOGY

Error → Fault → Failure — The Bug Chain



Testing's job: catch the fault (or its failure) as early in this chain as possible — the earlier, the cheaper to fix.

STLC — Six Phases, One Goal: Confidence



1. Requirement Analysis



2. Test Planning



3. Test Case Development



4. Environment Setup



5. Test Execution



6. Test Closure



STLC is a subset of the overall SDLC — it starts as soon as requirements are ready, running in parallel with development, not just at the end.

Requirement Analysis & Test Planning



1. Requirement Analysis

- QA team studies the SRS to identify what is testable
- Clarify ambiguous or missing requirements with stakeholders
- Identify types of testing needed (functional, performance...)
- Output: Requirement Traceability Matrix (RTM)



2. Test Planning

- Define scope, objectives, and testing approach
- Estimate effort, timeline, and resources needed
- Choose tools and define entry/exit criteria
- Output: Test Plan document

Test Case Development & Environment Setup



3. Test Case Development

- Write detailed test cases and test scripts
- Prepare test data required for execution
- Review and baseline test cases with the team
- Output: Test Cases, Test Data, Test Scripts



4. Test Environment Setup

- Set up hardware/software needed to run tests
- Configure servers, databases, and test data
- Perform a 'smoke test' to confirm environment readiness
- Output: Environment ready for execution

Test Execution & Test Closure



5. Test Execution

- Execute test cases against the build
- Log actual vs expected results
- Report defects (bugs) found back to developers
- Re-test fixed defects and track status



6. Test Closure

- Verify all planned test cases are executed
- Evaluate exit criteria and test coverage achieved
- Prepare Test Summary/Closure Report
- Document lessons learned for future cycles

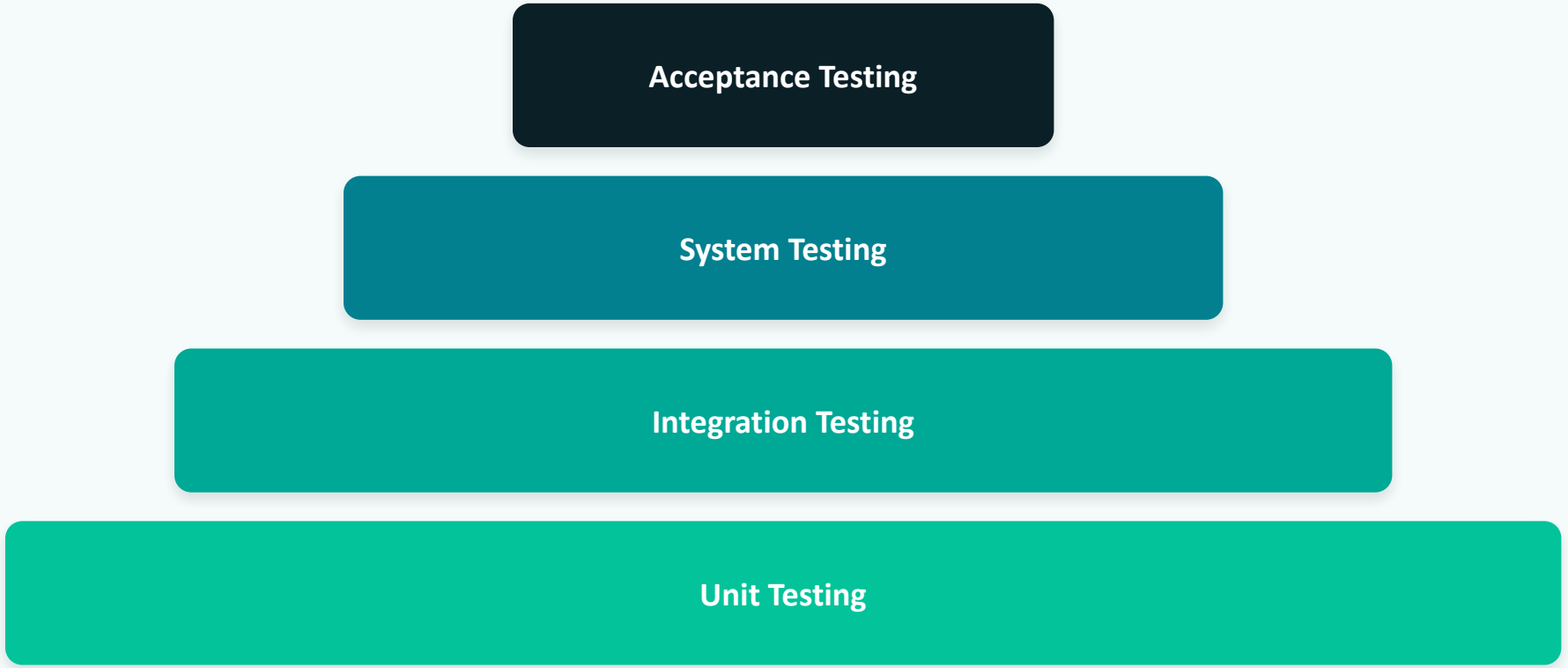


PART 3 OF 5

Testing Strategies — Levels of Testing

Unit → Integration → System → Acceptance → Regression

The Levels of Testing — Bottom to Top



Smaller, focused tests (Unit) run first and most often; broader tests (Acceptance) run later, closer to release.

Fixing a bug found in Unit Testing costs ₹; the same bug found in Acceptance Testing (or production) can cost ₹₹₹₹₹.

Unit Testing



Tests the smallest testable part of code — a single function, method, or class — in isolation.

Who does it?

The developer who wrote the code

When?

During/right after coding, before integration

Tools

JUnit (Java), PyTest (Python), NUnit (.NET)

Example

```
function add(a, b) {
  return a + b;
}

// Unit test
test("adds 2 + 3 to equal 5", () => {
  expect(add(2, 3)).toBe(5);
});
```

Testing a Discount Calculator Function

```
function calcDiscount(price, qty) {
  return qty >= 10 ? price * 0.9 : price;   // 10% off for bulk orders
}
```

Test Case	Input (price, qty)	Expected Output	Purpose
TC-01	100, 5	100	Below threshold — no discount
TC-02	100, 10	90	Exact boundary — discount applies
TC-03	100, 0	100	Edge case — zero quantity
TC-04	0, 15	0	Edge case — zero price

Integration Testing



Tests how two or more modules work together — catching bugs that only appear at the interfaces between units.



Example: The Login module works fine alone, and the Database module works fine alone — but integration testing catches the case where the login form sends a username in the wrong format the database expects, causing a crash.

- Focuses on data flow and communication between modules (APIs, DB calls, service calls)
- Performed after unit testing, once modules are individually verified
- Usually done by a dedicated test team, using stubs and drivers for missing modules

Integration Testing Approaches

Big Bang

All modules are combined at once and tested together. Simple, but hard to isolate defects.

Top-Down

Testing starts from the top-level module downward, using STUBS to simulate lower modules not yet built.

Bottom-Up

Testing starts from the lowest-level modules upward, using DRIVERS to simulate higher modules.

Sandwich (Hybrid)

Combines Top-Down and Bottom-Up — testing converges from both ends toward the middle.

System Testing



Tests the COMPLETE, integrated application as a whole — against the full set of requirements, in an environment close to production.



Functional

Does every feature work as specified?



Performance

Does it stay fast under load?



Security

Can it resist unauthorized access?



Usability

Is it easy and intuitive to use?

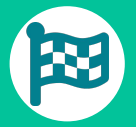
Example: Testing a complete e-commerce site end-to-end — browse → add to cart → checkout → payment → order confirmation email.

Acceptance Testing — Alpha vs Beta



Alpha Testing

- Performed by INTERNAL staff (in-house QA / employees)
- Conducted at the developer's site
- Happens before the software is released externally
- Goal: catch major issues before real users see it



Beta Testing

- Performed by REAL/external end users, in their own environment
- Conducted after alpha testing, before final release
- Feedback gathered from a limited public release
- Goal: validate real-world usability and catch edge cases

Regression Testing



Re-running existing test cases after a code change, to make sure the change didn't break anything that used to work.



Real example: A developer fixes a bug in the 'Apply Coupon' feature. Regression testing re-runs the checkout, cart total, and tax calculation test cases too — to confirm the fix didn't silently break the tax calculation.

- Triggered by: bug fixes, new features, configuration changes
- Often automated, since the same tests run repeatedly across builds
- Critical part of CI/CD pipelines — runs on every code commit

All Testing Levels, Side by Side

Level	Performed By	When	Focus
Unit	Developer	During coding	Single function/module
Integration	Testers/Devs	After unit testing	Interfaces between modules
System	QA Team	After integration	Whole application vs requirements
Acceptance	Users/Client	Before release	Business needs & usability
Regression	Testers (often automated)	After any change	Nothing old broke



PART 4 OF 5

Testing Techniques

Black-Box vs White-Box — two lenses for designing tests

Black-Box Testing



Tests functionality WITHOUT looking at the internal code — the tester only knows inputs and expected outputs, like a sealed black box.



Performed by

Testers (may not know how to code)



Based on

Requirements & specifications



Common Techniques

Equivalence Partitioning, Boundary Value Analysis, Decision Tables

BLACK-BOX TECHNIQUE 1

Equivalence Partitioning — Worked Example

Idea: Split all possible inputs into groups ("partitions") that should behave the same way — then test just ONE value from each group.

Scenario: An age field accepts values 18–60 for a job application.

Invalid (Low)

$Age < 18$

Test value: e.g. 10

Valid

$18 \leq Age \leq 60$

Test value: e.g. 35

Invalid (High)

$Age > 60$

Test value: e.g. 75

Result: instead of testing 100+ possible ages, we only need 3 representative test cases — one per partition.

BLACK-BOX TECHNIQUE 2

Boundary Value Analysis — Worked Example

Idea: Most bugs hide at the EDGES of valid ranges, not the middle — so test values right at, just below, and just above each boundary.

Same scenario: Age field accepts values 18–60.

Test Value	Position	Expected Result
17	Just below lower boundary	Rejected (Invalid)
18	Lower boundary	Accepted (Valid)
19	Just above lower boundary	Accepted (Valid)
59	Just below upper boundary	Accepted (Valid)
60	Upper boundary	Accepted (Valid)
61	Just above upper boundary	Rejected (Invalid)

White-Box Testing



Tests the INTERNAL structure, logic, and code paths — the tester can see (and needs to understand) the source code, like looking through a clear box.



Performed by

Developers / testers who can read code



Based on

Internal logic, code paths, conditions



Common Techniques

Statement Coverage, Branch Coverage, Path Coverage, Cyclomatic Complexity

Statement & Branch Coverage — Example

```
function checkGrade(marks) {
  if (marks >= 40) {
    return "Pass";
  } else {
    return "Fail";
  }
}
```

Statement Coverage

Ensures every LINE of code executes at least once. Here: marks=50 alone covers the Pass line but NOT the Fail line — 50% statement coverage.

Branch Coverage

Ensures every TRUE/FALSE branch of a condition executes. Needs BOTH marks=50 (True) AND marks=30 (False) for 100% branch coverage.

Test Case	Input (marks)	Branch Taken	Output
TC-01	50	if TRUE	Pass
TC-02	30	else FALSE	Fail

Cyclomatic Complexity — Path Coverage

Formula

$$V(G) = E - N + 2$$

E = number of edges, N = number of nodes in the control-flow graph.

V(G) tells you the MINIMUM number of independent test paths needed to cover the code's logic.

Rule of thumb: V(G) ≤ 10 ⟶ manageable
V(G) > 10 ⟶ code is too complex, consider refactoring

Worked Example

```
if (a > b) {
  max = a;
} else {
  max = b;
}
if (max > c) {
  max = max;
} else {
  max = c;
}
```

2 decision points (if statements) ⟶ 2 predicate nodes
V(G) = Number of decisions + 1 = 2 + 1 = 3

⟶ 3 independent paths needed to fully test this function's logic.

Black-Box vs White-Box Testing

Aspect	Black-Box Testing	White-Box Testing
Knowledge Needed	No code knowledge required	Requires code/logic knowledge
Basis	Requirements & specifications	Internal code structure
Performed By	Testers, end-users	Developers, test engineers
Techniques	Equivalence Partitioning, BVA	Statement/Branch/Path Coverage
Also Known As	Behavioral / Functional testing	Structural / Glass-box testing



PART 5 OF 5

Test Case Design

Turning technique into a concrete, traceable test artifact

What Is a Test Case? — Core Components

> *"A documented set of steps, inputs, and expected results, used to verify that a specific part of the software works correctly."*



Test Case ID
Unique identifier, e.g. TC-Login-01



Description
What is being tested and why



Preconditions
State the system must be in first



Test Steps
Exact, ordered actions to perform



Test Data
Specific input values to use



Expected Result
What SHOULD happen



Actual Result
What DID happen (filled during execution)



Status
Pass / Fail / Blocked

Sample Test Case: Login Form

Field	Value
Test Case ID	TC-Login-05
Description	Verify login fails with an incorrect password
Precondition	User account 'raj@example.com' already exists
Test Steps	1) Open login page 2) Enter valid email 3) Enter wrong password 4) Click 'Login'
Test Data	email: raj@example.com, password: WrongPass123
Expected Result	Error message: 'Invalid email or password' is shown; user stays on login page
Actual Result	(filled in during test execution)
Status	Pass ✓

Best Practices for Writing Test Cases

Keep It Atomic
One test case should verify ONE specific condition or behavior.

Make It Traceable
Link each test case back to a specific requirement (via RTM).

Cover Edge Cases
Don't just test the 'happy path' — include invalid & boundary inputs.

Write for Others
Anyone on the team should be able to execute it without asking you.

Keep It Reusable
Design so it can be re-run easily for regression testing.

Be Specific
Use exact data and steps, never vague instructions like 'test login'.



REAL-WORLD CASE STUDY

The Ariane 5 Rocket — 37 Seconds to Failure

In 1996, the Ariane 5 rocket self-destructed 37 seconds after launch — destroying a $370M payload. The cause: reused navigation software from Ariane 4 was never re-tested for Ariane 5's higher flight speeds. A 64-bit number conversion to 16-bit overflowed, an unhandled error crashed the guidance system, and the rocket veered off course.

Lesson: even code that "worked before" must be re-tested in a new context.

What Testing Would Have Caught It

- White-box testing on the reused module with Ariane 5's new flight parameters
- Boundary value analysis on the numeric conversion
- Regression testing after reusing legacy code in a new context
- Proper code review of untested, reused modules

Everything We Covered, at a Glance



Coding Standards & Review



STLC (6 phases)

SOFTWARE CODING & TESTING



Test Case Design



Testing Strategies (Unit→Regression)



Black-Box & White-Box

TEST YOUR KNOWLEDGE

Quick Quiz — Unit 5



Which testing type checks 'Are we building the right product?'

→ *Validation*



Which black-box technique tests values right at a range's edge?

→ *Boundary Value Analysis*



Formula for Cyclomatic Complexity?

→ *V(G) = E − N + 2*



Testing performed by real end-users before release?

→ *Beta Testing*



Testing re-run after a bug fix to check nothing else broke?

→ *Regression Testing*



Thank You!

Questions? Let's discuss Software Coding & Testing.